



Modelización en Ingeniería Aeroespacial

Clase 6: Discretización de Modelos y Estabilidad Numérica

Ing. Jair Molina Arce

Universidad Internacional de Innovación

11 de enero de 2026

Agenda

- ¿Por qué discretizar modelos continuos?
- Modelos en tiempo continuo vs tiempo discreto
- Discretización de sistemas lineales (exacta/ZOH)
- Métodos numéricos como discretización (Euler, RK)
- Estabilidad numérica y regiones de estabilidad
- **Ejemplos con señales (continuo vs discreto)**
- **Práctica: señal amortiguada (continua y discretizada)**
- **Actividad: discretiza una señal propuesta y explora parámetros**



Motivación

¿Por qué discretizamos?

Objetivo

Entender cómo pasar de ecuaciones diferenciales a ecuaciones en diferencias sin destruir la dinámica física del sistema.

Realidad ingenieril

- Computadoras trabajan en **tiempo discreto**
- Sensores muestrean a intervalos T_s
- Controladores digitales (MCU, FPGA, DSP)

Idea clave

Un sistema estable en continuo puede volverse **inestable** al discretizarlo mal.



Modelos continuos vs discretos

Forma estándar

$$\dot{x}(t) = f(x(t), u(t))$$

- Tiempo continuo $t \in \mathbb{R}$
- Solución exacta rara vez disponible
- Base física del sistema

Ejemplo aeroespacial

Dinámica de vuelo, actitud, órbitas, combustión, vibraciones.

Forma general

$$x_{k+1} = F(x_k, u_k)$$

- Tiempo discreto kT_s
- Aproximación del modelo continuo
- Dependiente del método numérico

Advertencia

$F(\cdot)$ **no es único**: depende del método de discretización.



Discretización de sistemas lineales

Modelo

$$\dot{x} = Ax + Bu$$

Queremos:

$$x_{k+1} = A_d x_k + B_d u_k$$

Pregunta central

¿Cómo obtener A_d , B_d sin perder estabilidad?

Discretización exacta (ZOH)

Entrada constante por intervalo ($u(t) = u_k$)

$$A_d = e^{AT_s}$$
$$B_d = \int_0^{T_s} e^{A\tau} B d\tau$$

- Preserva estabilidad y dinámica local
- Es la forma correcta para control digital con ZOH

Regla

Si $\Re(\lambda_i(A)) < 0$ entonces típicamente $|\lambda_i(A_d)| < 1$.



Métodos numéricos como discretización

Aproximación

$$x_{k+1} = x_k + T_s(Ax_k + Bu_k)$$

$$A_d = I + T_s A$$

- Muy simple
- Puede ser inestable para pasos grandes

Advertencia

Euler explícito puede volver inestable un sistema estable.

Ejemplo: estable continuo $\rightarrow$ inestable discreto

Sistema continuo

$$\dot{x} = -2x$$

Continuo: $\lambda = -2$ (estable).

Euler

$$x_{k+1} = (1 - 2T_s)x_k$$

Condición de estabilidad

$$|1 - 2T_s| < 1 \Rightarrow 0 < T_s < 1$$

Si $T_s > 1$, el método diverge aunque el sistema sea estable.



Regiones de estabilidad

Criterio general

Para el modo $\dot{x} = \lambda x$, el método es estable si:

$$|R(z)| < 1, \quad z = \lambda T_s$$

- $R(z)$: función de estabilidad del método
- Estabilidad depende de λ y del paso T_s

Lectura física

No basta con que λ sea negativo: importa qué tanto “cabe” en el paso.

Resumen (intuición)

- **Euler explícito:** región pequeña (peligroso en modos rápidos)
- **RK4:** región más grande (robusto para muchos sistemas)
- **Implícitos:** región grande (ideal para sistemas stiff)

Regla de ingeniería

Si el sistema tiene modos rápidos:

- reduce T_s o
- cambia el integrador



Ejemplos con señales

Ejemplo 1 (1/2): seno continuo vs muestreado

Señal continua

$$x(t) = A \sin(2\pi f t)$$

Señal discretizada (muestreo)

$$x[k] = x(kT_s) = A \sin(2\pi f k T_s)$$

- Si f es alta y T_s es grande, aparecen distorsiones (aliasing).
- En control digital y sensores: el muestreo define lo que **ves** del fenómeno.

Tip

Usa $f_s = \frac{1}{T_s}$ suficientemente alto respecto a f para representar bien la señal.

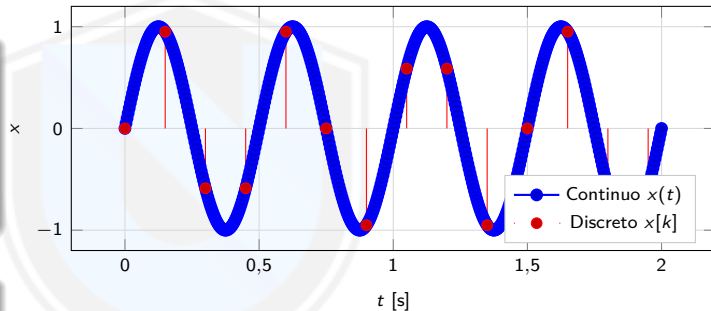
Ejemplo 1 (2/2): gráfica seno continuo vs muestreado

Parámetros (demo)

- $A = 1$
- $f = 2 \text{ Hz}$
- $T_s = 0,15 \text{ s}$
- $t \in [0, 2] \text{ s}$

Lectura

Marcadores separados $\Rightarrow$ pocos puntos por ciclo.



Ejemplo 2 (1/2): exponencial (decaimiento) continuo vs discreto

Señal continua

$$x(t) = Ae^{-at}, \quad a > 0$$

Muestreo directo

$$x[k] = Ae^{-akT_s}$$

- Es un caso muy limpio: el muestreo **no cambia** la estabilidad (solo la resolución).
- Sirve como referencia para entender señales amortiguadas.

Idea

Aquí no hay oscilación: solo una escala de tiempo $\tau = \frac{1}{a}$.

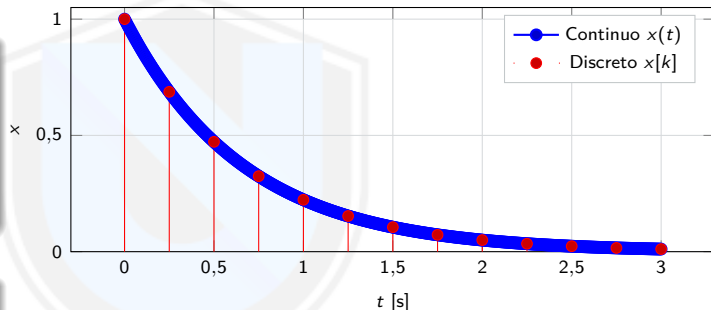
Ejemplo 2 (2/2): gráfica exponencial continuo vs muestreado

Parámetros (demo)

- $A = 1$
- $a = 1,5 \text{ s}^{-1}$
- $T_s = 0,25 \text{ s}$
- $t \in [0, 3] \text{ s}$

Lectura

La forma se conserva; cambian los puntos disponibles.



Práctica: señal amortiguada (continua y discretizada)

Práctica (1/2): señal amortiguada (modelo continuo y muestreo)

Objetivo

Comparar una señal amortiguada en continuo vs su versión discretizada.

Señal (oscilación amortiguada)

$$x(t) = A e^{-\alpha t} \sin(2\pi f t + \varphi)$$

Muestreo ideal

$$x[k] = x(kT_s) = A e^{-\alpha k T_s} \sin(2\pi f k T_s + \varphi)$$

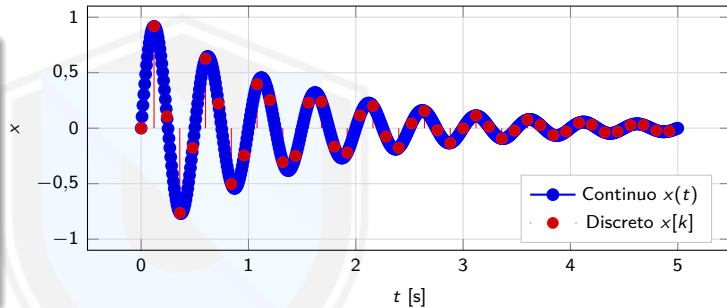
- A : amplitud inicial
- α : amortiguamiento (1/s)
- f : frecuencia (Hz)
- φ : fase (rad)

Lectura física

Práctica (2/2): gráfica señal amortiguada continuo vs muestreado

Parámetros (demo)

- $A = 1$
- $\alpha = 0,7 \text{ s}^{-1}$
- $f = 2 \text{ Hz}$
- $\varphi = 0$
- $T_s = 0,12 \text{ s}$
- $t \in [0, 5] \text{ s}$



Objetivo

Elegir T_s para conservar forma, fase y envolvente.



Código Python de la práctica

Estructura del script (visión general)

Bloques del programa

- 1 Parámetros (A , α , f , ϕ , T_s , duración)
- 2 Generación de malla continua t (alta resolución)
- 3 Generación de instantes discretos $t_k = kT_s$
- 4 Cálculo $x(t)$ y $x[k]$
- 5 Gráficas: continuo (línea) + discreto (stems o marcadores)

Tip de ingeniería

Mismo modelo físico, dos representaciones: lo que cambia es el **tiempo** (continuo vs muestreado).

Código Python: señales y muestreo

Funciones principales

`damped_signal(t, A, alpha, f, phi)`

Calcula: $x(t) = A \cdot e^{-\alpha t} \cdot \sin(2\pi f t + \varphi)$

`sample_times(Ts, t0, tf)`

Genera instantes discretos: $t_k = t_0 + k \cdot T_s$

`main()`

Configura parámetros, calcula señales y grafica

Código disponible

El código Python completo está disponible en el repositorio del curso. Incluye NumPy, Matplotlib y ejemplos completos para experimentar con diferentes parámetros.



Actividad

Actividad (1/2): señal propuesta para discretizar

Objetivo

Cada alumno debe discretizar la señal, graficar continuo vs discreto y justificar el T_s elegido.

Señal propuesta (chirp lineal amortiguado)

$$x(t) = A e^{-\alpha t} \sin\left(2\pi\left(f_0 t + \frac{1}{2}\beta t^2\right) + \varphi\right)$$

- f_0 : frecuencia inicial (Hz)
- β : tasa de barrido (Hz/s)
- Misma idea: $x[k] = x(kT_s)$

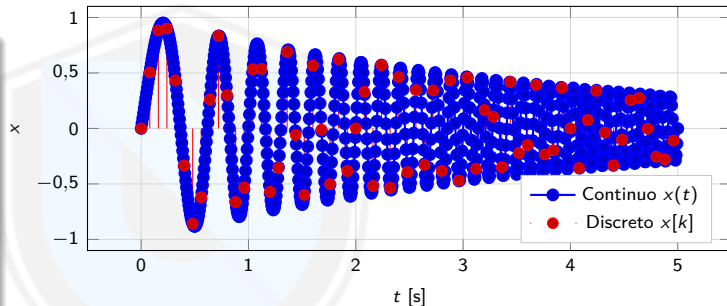
Qué deben modificar

$A, \alpha, f_0, \beta, \varphi, T_s$ y observar aliasing vs fidelidad.

Actividad (2/2): gráfica ejemplo (chirp amortiguado) continuo vs muestreado

Parámetros (demo)

- $A = 1$
- $\alpha = 0,25 \text{ s}^{-1}$
- $f_0 = 1 \text{ Hz}$
- $\beta = 2,0 \text{ Hz/s}$
- $\varphi = 0$
- $T_s = 0,08 \text{ s}$
- $t \in [0, 5] \text{ s}$



Lectura

A mayor t , sube $f(t) = f_0 + \beta t$; si f_s es bajo, aparece aliasing.

Sistemas rígidos (Stiff systems)

¿Qué es un sistema stiff?

Definición práctica

Sistema con escalas de tiempo muy distintas:

$$|\Re(\lambda_{\text{rápido}})| \gg |\Re(\lambda_{\text{lento}})|$$

- La estabilidad obliga a pasos muy pequeños
- La precisión no es el problema: la estabilidad numérica sí

Ejemplos aeroespaciales

- combustión (rápida) + estructura (lenta)
- térmico (lento) + actuadores (rápidos)
- vibración local (rápida) + trayectoria (lenta)

Qué se observa en la práctica

Síntomas típicos

- La simulación “explota” con Euler, aunque la física sea estable
- RK4 requiere pasos muy pequeños para no divergir
- El tiempo de cómputo se dispara

Soluciones

- Integradores implícitos (Backward Euler, BDF)
- RK adaptativos (solve_ivp con métodos stiff: Radau, BDF)
- Separación de escalas / reducción de modelo

Consecuencias en control digital

Realidad de implementación

- Control diseñado en continuo
- Implementado en discreto con T_s
- ZOH y discretización afectan polos

Regla empírica

Para capturar dinámica sin distorsión severa:

$$T_s \leq \frac{1}{10} \tau_{\text{modo más rápido}}$$

Mensaje final: simulación vs implementación

Simulación

- objetivo: entender física
- tolera pasos pequeños
- comparas integradores

Implementación (MCU)

- objetivo: ejecutar rápido y estable
- T_s limitado por hardware
- discretización correcta es crítica

Regla

Si el resultado cambia drásticamente al cambiar T_s , aún no dominas el modelo.



Resumen

- 1 Discretizar es una decisión física y numérica
- 2 Euler explícito puede destruir estabilidad
- 3 La estabilidad numérica depende de $z = \lambda T_s$
- 4 Los sistemas stiff requieren métodos adecuados
- 5 Muestreo T_s define fidelidad de señales y control digital

Siguiente clase:

Control lineal: feedback, polos y diseño (introducción)

¿Preguntas?

`jair.molina@unii.edu.mx`

Material disponible en el repositorio del curso